INDIRA COLLEGE OF ENGINEERING & MANAGEMENT
AN AUTONOMOUS INSTITUTE
AFFILIATED TO SAVITRIBAI PHULE PUNE UNIVERSITY (SPPU)



Department of Artificial Intelligence and Data Science

Lab Manual

CRISPER Learning for RPA Lab

2026-2027

Indira College of Engineering and Management, Pune

Department of Artificial Intelligence and Data Science



Laboratory Manual

Academic Year: 2026-27
(Semester –V)

Course: CRISPER Learning for RPA Lab

Course Code: 24UAIP5M13

Course Coordinator: Prof. Trupti Bhagat

List of Laboratory Experiments

List of Assignments/Practical's:

Sr. No.	Title of Practical
1	Automate Excel data processing by reading data from one sheet and generating a formatted output sheet using Python or automation tools.
2	Develop an automated email notification system that reads recipient details from an Excel file and sends emails automatically.
3	Perform web data extraction from a website (e.g., news headlines or product prices) and store the extracted data in Excel or CSV format.
4	Implement an automated file organization system that sorts files into folders based on file type (PDF, images, documents).
5	Automate attendance data processing by calculating attendance percentage and generating a summary report from a dataset.
6	Perform automated data cleaning by handling missing values, removing duplicate records, and generating a cleaned dataset.
7	Generate automated summary reports from a dataset (e.g., sales or student marks data) using Python.
8	Analyze automation performance by comparing manual processing time with automated processing time and calculating time savings.
9	Develop a simple web form automation that reads input data from an Excel sheet and automatically fills and submits the form.
10	Create an automation workflow that reads data from a dataset and generates visual charts or graphs for analysis.
Mini Project (Any 1 Compulsory)	
11	Mini Project: Develop an automated system for student result processing including data extraction, calculation of results, and report generation.
12	Mini Project: Build an automated leave approval workflow that processes requests, sends notifications, and records approvals.
13	Mini Project: Develop a web data monitoring bot that periodically extracts information from websites and generates trend reports.
14	Mini Project: Develop a complete automation workflow for a real-world process such as student result processing or automated report generation.

@The CO-PO mapping table

PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CO1	3	2	2	1	3	-	-	-	-	-	-	1
CO2	2	3	2	3	2	1	-	1	2	2	1	1
CO3	1	2	3	2	2	1	-	1	2	3	1	1
CO4	2	3	2	3	2	2	1	1	1	2	3	2
CO5	-	-	-	-	-	-	-	-	-	-	-	-

Experiment No: 1

Title

Automating Excel Data Processing and Generating a Formatted Output Sheet Using Python

Aim

To automate Excel data processing by reading data from one worksheet and generating a formatted output worksheet using Python.

Objectives

- To understand Excel automation using Python.
- To learn how to read and write Excel files programmatically.
- To process and format worksheet data automatically.
- To generate automation-ready reports.

Theory

Excel automation is widely used in business environments to reduce manual effort and improve data accuracy. Python libraries such as **pandas** and **openpyxl** help in reading, processing, and formatting Excel data efficiently.

Requirements

- Python
- pandas library
- openpyxl library
- Excel file dataset

Procedure

1. Install required Python libraries.
2. Load the Excel workbook.
3. Read data from the source sheet.
4. Process and format the data.
5. Generate a new output worksheet.
6. Save the workbook.

Python Code

```
import sys
print(sys.executable)
import pandas as pd
# 1. Read Excel file
data = pd.read_excel(r"C:\Users\Admin\Downloads\input.xlsx")

# 2. Clean column names (VERY IMPORTANT)
data.columns = (
    data.columns
    .str.strip()
    .str.replace('\ufeff', '', regex=True)
    .str.replace(" ", "", regex=False)
)

# 3. Show columns for confirmation
print("Columns in file:", data.columns.tolist())

# 4. Check required columns safely
if "Marks_1" in data.columns and "Marks_2" in data.columns:

    # 5. Calculate Total
    data["Total"] = data["Marks_1"] + data["Marks_2"]

    # 6. Save output file
    output_path = r"C:\Users\Admin\Downloads\output.xlsx"
    data.to_excel(output_path, index=False)

    # 7. Show result in console
```

```
print("\n--- OUTPUT DATA ---")
    print(data)


    print("\nFile saved successfully at:")
    print(output_path)


else:
    print("\nERROR: Required columns not found!")
    print("Available columns are:")
    print(data.columns.tolist())
```

Step 1: Import pandas Library

```
import pandas as pd
```

Explanation

- pandas is a Python library used for data handling and Excel processing.
- pd is an alias for pandas.

Output

No output is displayed.

Step 2: Read Excel File

Suppose input.xlsx contains the following data:

Name	Marks1	Marks2
Amit	78	82
Neha	65	70
Raj	90	88

Code:

```
data = pd.read_excel("input.xlsx")
```

Explanation

- Reads data from the Excel file.
- Stores it in a DataFrame called data.

Output (DataFrame)

```
print(data)
```

Output:

Sr. No.	Name	Marks1	Marks2
1	Amit	78	82
2	Neha	65	70
3	Raj	90	88

Step 3: Create Total Column

Code:

```
data["Total"] = data["Marks1"] + data["Marks2"]
```

Explanation

- Adds values of Marks1 and Marks2.
- Stores result in a new column named Total.

Output

```
print(data)
```

Output:

Sr. No.	Name	Marks1	Marks2	Total
1	Amit	78	82	160
2	Neha	65	70	135
3	Raj	90	88	178

Step 4: Save Output to Excel

Code:

```
data.to_excel("output.xlsx", index=False)
```

Explanation

- Saves processed data into a new Excel file named output.xlsx.
- index=False removes row numbering from Excel.

Output File (output.xlsx)

Name	**Marks1**	**Marks2**	**Total**
Amit	78	82	160
Neha	65	70	135
Raj	90	88	178

Step 5: Display Success Message

Code:

```
print("Formatted output sheet generated successfully.")
```

Output

Formatted output sheet generated successfully.

Observation

- Data was successfully read from the source sheet.
- A formatted output sheet was generated automatically.

Conclusion: Excel data processing automation was successfully implemented using Python.

Experiment No: 2

Title:

Develop an automated email notification system that reads recipient details from an Excel file and sends emails automatically.

Aim:

To develop an automated email notification system that reads recipient details from an Excel file and sends emails automatically.

Objectives:

1. To understand the email automation.
2. To understand the libraries like Requirments, MIMEText and pathlib.
3. To understand how to generate a mail automatically.

Theory:

Developing an automated email notification system from Excel requires three core layers: data extraction, message processing, and email transmission. The system reads structured recipient details, maps variables to email templates, and dispatches messages programmatically via an SMTP server, cloud services, or local email applications.

Requirement:

1. Pycharm IDE
2. Packages like requirements
3. Desktop

Project Structure

```
email_automation/
│
├── recipients.xlsx
├── send_emails.py
├── email_template.txt
└── requirements.txt
```

how to create new py file using command promp in this file path
C:\Users\Admin\Documents\MyPythonProject

You can create a new .py file directly from **Command Prompt** using simple commands.

Your path:

C:\Users\Admin\Documents\MyPythonProject

```
C:\Users\Admin>cd C:\Users\Admin\Documents\MyPythonProject

C:\Users\Admin\Documents\MyPythonProject>echo. > send_emails.py

C:\Users\Admin\Documents\MyPythonProject>type nul > send_emails.py

C:\Users\Admin\Documents\MyPythonProject>notepad send_emails.py

C:\Users\Admin\Documents\MyPythonProject>dir
 Volume in drive C has no label.
 Volume Serial Number is 4E38-2910

 Directory of C:\Users\Admin\Documents\MyPythonProject

28-05-2026  12:14    <DIR>          .
28-05-2026  12:08    <DIR>          ..
28-05-2026  12:08    <DIR>          .spyproject
28-05-2026  12:09                 0 Requirments.txt
28-05-2026  12:14                 0 send_emails.py
               2 File(s)              0 bytes
               3 Dir(s)  107,701,956,608 bytes free

C:\Users\Admin\Documents\MyPythonProject>
```

Step 1: Open Command Prompt

Press:

Win + R → type cmd → Enter

Step 2: Go to your project folder

Run this:

cd C:\Users\Admin\Documents\MyPythonProject

Step 3: Create a new Python file

Option 1: using echo (easiest)
Command: echo. > send_emails.py

This creates an empty file named send_emails.py

Option 2: using type nul (better)
Command: type nul > send_emails.py

Option 3: create + open in Notepad
Command: notepad send_emails.py

Opens file immediately for editing

Step 4: Verify file created

Type command: dir

You will see:

send_emails.py

Step 1: Install Dependencies

Create requirements.txt

```
pandas
openpyxl
```

Install:

```
pip install -r requirements.txt
```

Step 2: Create Excel File

Create recipients.xlsx

Name	Email	Status
John Doe	john@example.com	Pending
Jane Smith	jane@example.com	Pending

Step 3: Create Email Template

Create email_template.txt

```
Hello {name},

This is an automated notification email.

Your current status is: {status}

Regards,
Automation System
```

Step 4: Python Automation Script

Create send_emails.py

```
import os
SENDER_PASSWORD = os.getenv("GMAIL_APP_PASSWORD")
import pandas as pd
import smtplib
```

```
from email.mime.text import MIMEText


from email.mime.multipart import MIMEMultipart
from pathlib import Path

# ==========================
# EMAIL CONFIGURATION
# ==========================
SMTP_SERVER = "smtp.gmail.com"
SMTP_PORT = 587

SENDER_EMAIL = "dhokevidya27@gmail.com"
SENDER_PASSWORD = "hhxe atto vupb bnma"

SUBJECT = "Automated Notification"

# ==========================
# LOAD EXCEL FILE
# ==========================
excel_file = "recipients.xlsx"

df = pd.read_excel(excel_file)

# ==========================
# LOAD EMAIL TEMPLATE
# ==========================
template_path = Path("email_template.txt")
email_template = template_path.read_text()

# ==========================
# CONNECT SMTP SERVER
# ==========================
server = smtplib.SMTP(SMTP_SERVER, SMTP_PORT)
server.starttls()

server.login(SENDER_EMAIL, SENDER_PASSWORD)

# ==========================
# SEND EMAILS
# ==========================
for index, row in df.iterrows():

    recipient_name = row["Name"]
    recipient_email = row["Email"]
    recipient_status = row["Status"]

    # Customize email body
    body = email_template.format(
        name=recipient_name,
        status=recipient_status
    )

    # Create message
    message = MIMEMultipart()
```

```
    message["From"] = SENDER_EMAIL
    message["To"] = recipient_email


 message["Subject"] = SUBJECT

    message.attach(MIMEText(body, "plain"))

    try:
        server.sendmail(
            SENDER_EMAIL,
            recipient_email,
            message.as_string()
        )

        print(f"Email sent to {recipient_email}")

    except Exception as e:
        print(f"Failed to send email to {recipient_email}")
        print(e)

# ============================
# CLOSE CONNECTION
# ============================
server.quit()

print("All emails processed.")
```

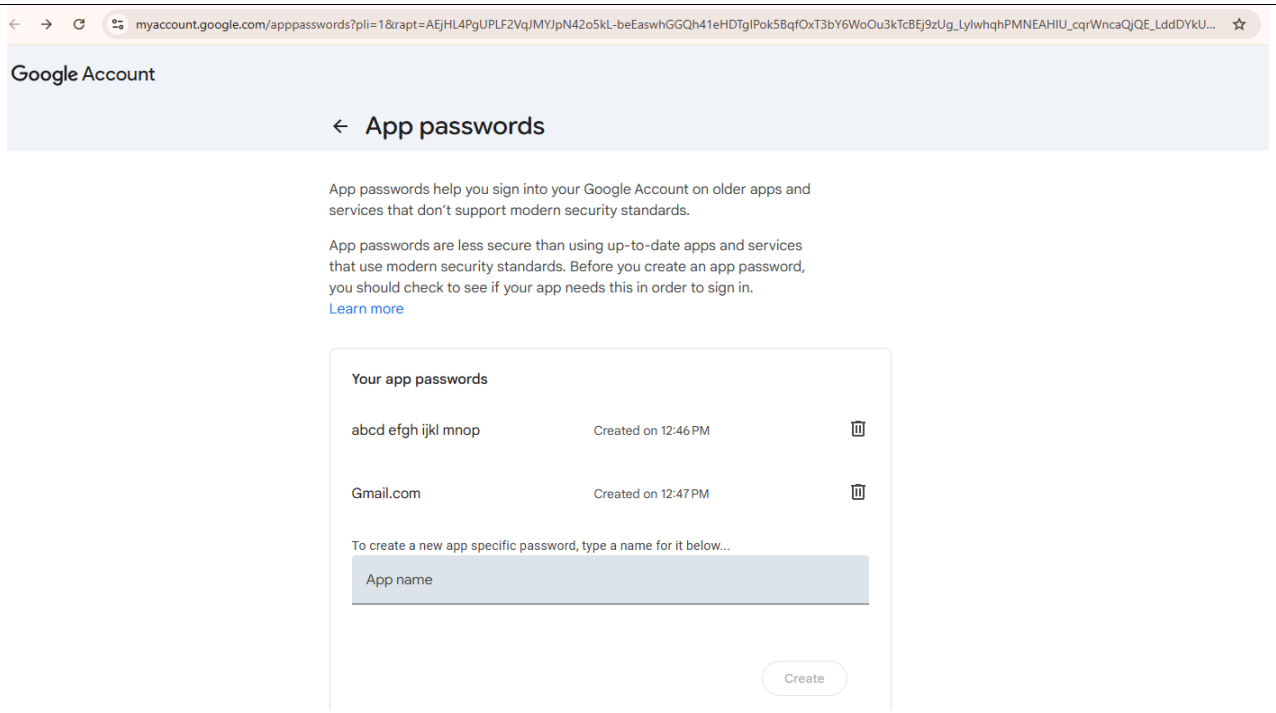
Step 5: Gmail Setup

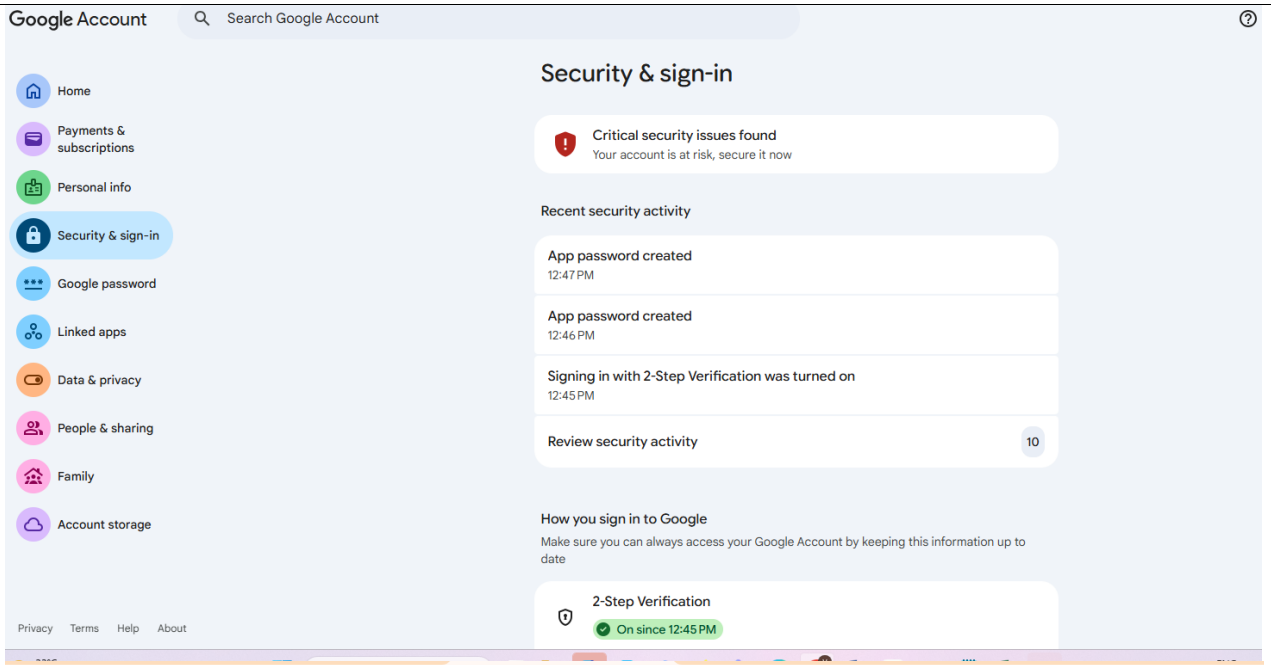
For Gmail:

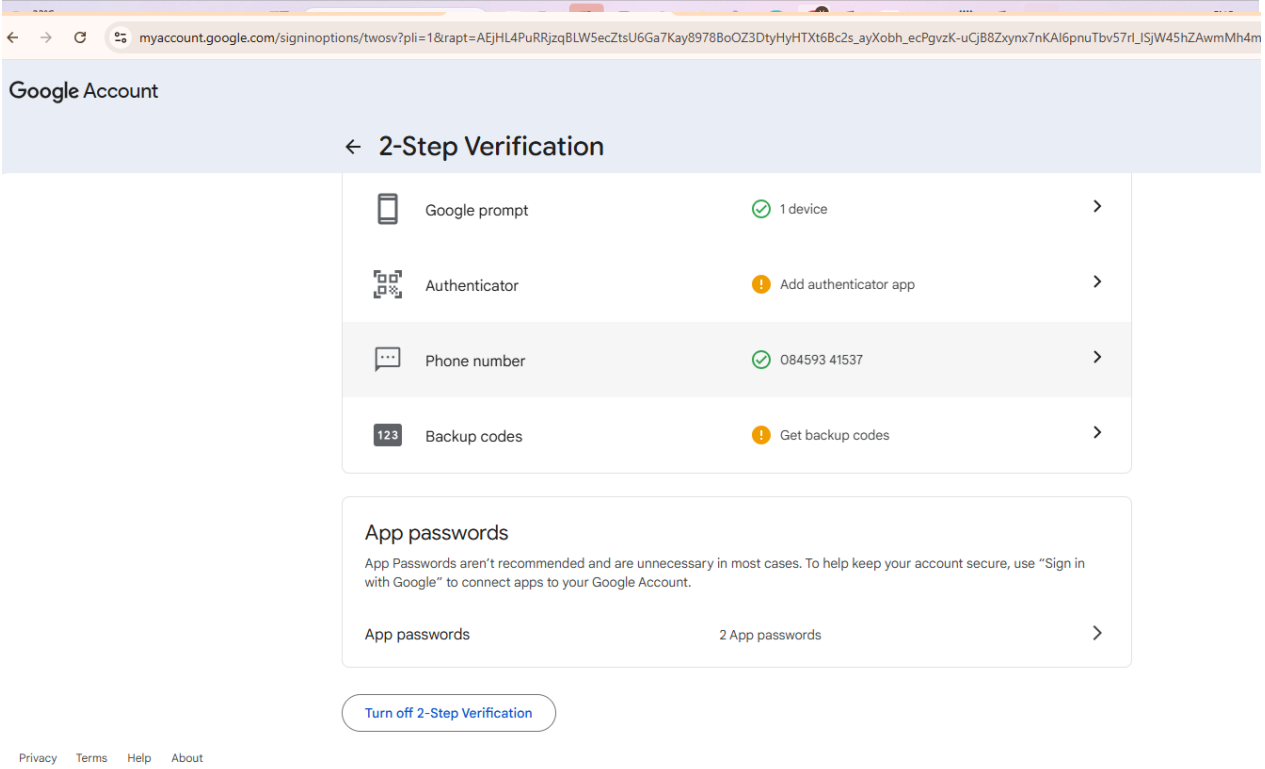
1. Enable 2-Step Verification
2. Generate App Password

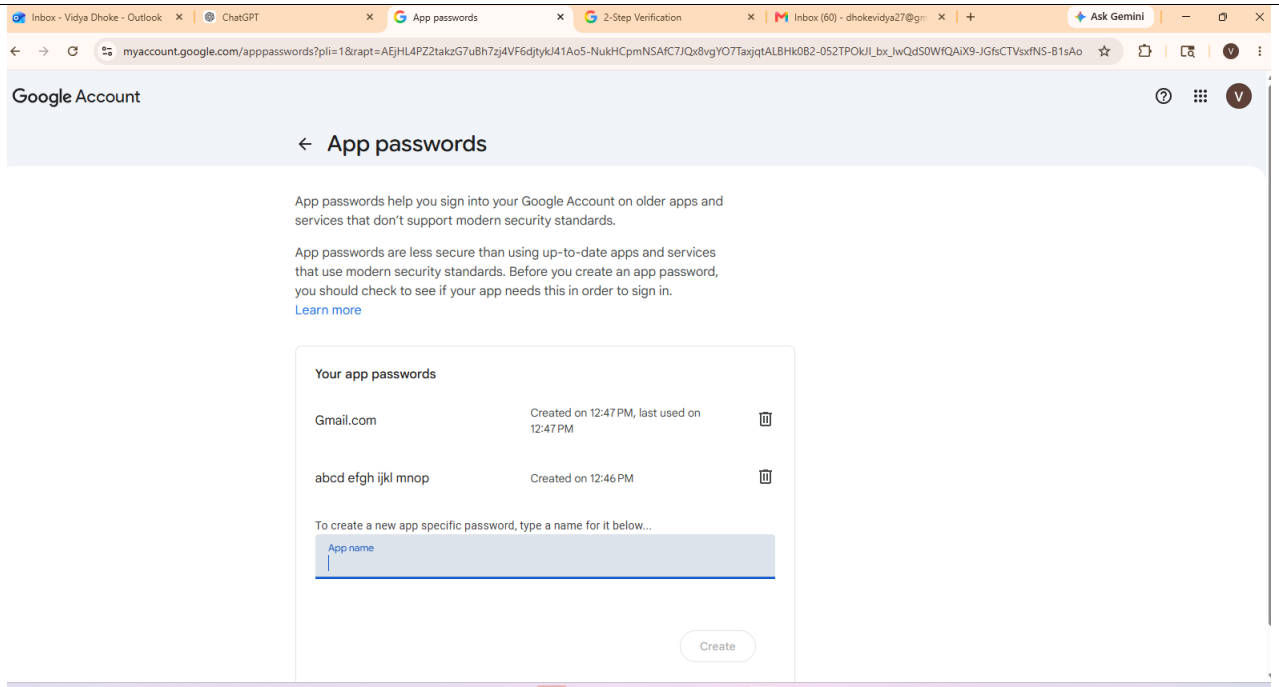
Use the generated app password instead of your Gmail password.

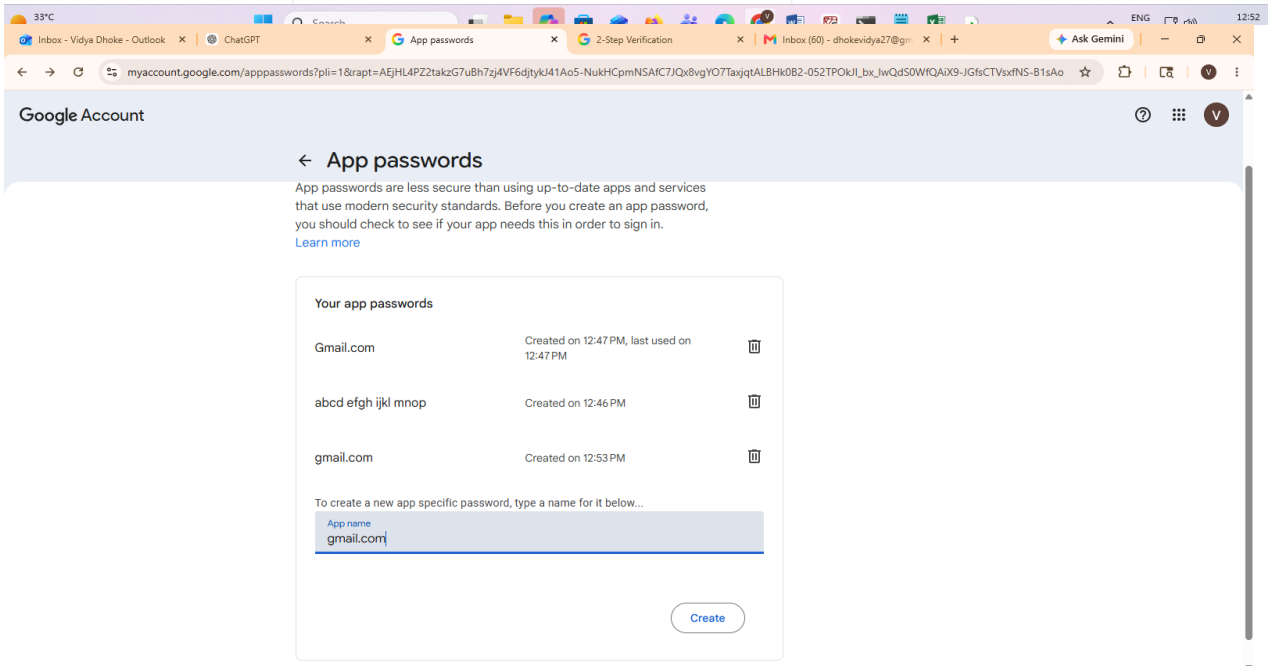
Google Account → Security → App Passwords


myaccount.google.com/apppasswords?pli=1&rapt=AEjHL4PgUPLF2VqJMYJpN42o5kL-beEaswhGGQh41eHDTgIPok5BqfOxT3bY6WoOu3kTcBEj9zUg_LyIwhqhPMNEAHIU_cqrWncaQjQE_LddDYkU...
Google Account
App passwords
App passwords help you sign into your Google Account on older apps and services that don't support modern security standards.
App passwords are less secure than using up-to-date apps and services that use modern security standards. Before you create an app password, you should check to see if your app needs this in order to sign in.
Learn more
Your app passwords
abcd efgh ijkl mnop
Created on 12:46 PM
Gmail.com
Created on 12:47 PM
To create a new app specific password, type a name for it below...
App name
Create

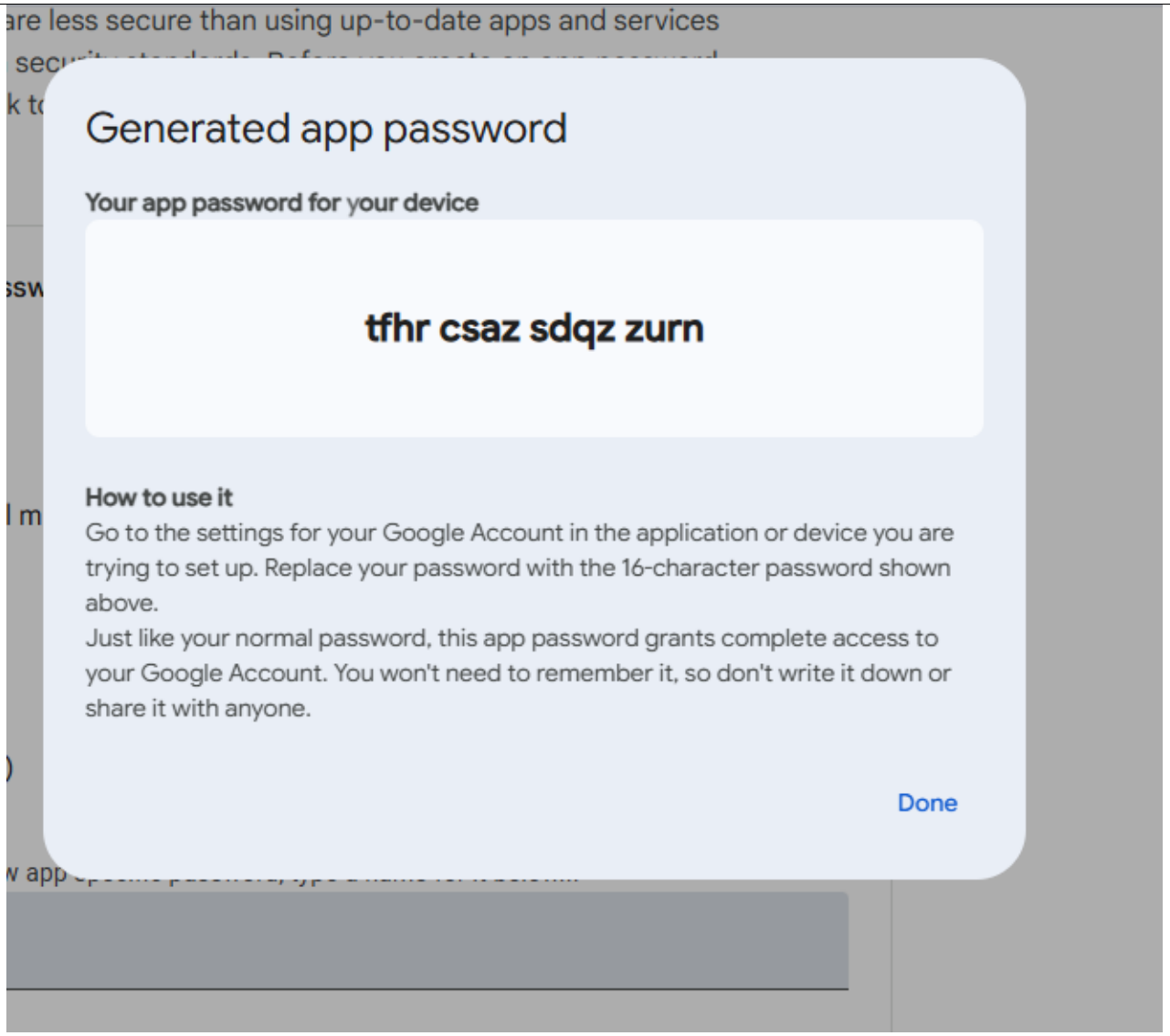

Google Account
Search Google Account
Home
Payments & subscriptions
Personal info
Security & sign-in
Google password
Linked apps
Data & privacy
People & sharing
Family
Account storage
Privacy Terms Help About
Security & sign-in
Critical security issues found
Your account is at risk, secure it now
Recent security activity
App password created
12:47 PM
App password created
12:46 PM
Signing in with 2-Step Verification was turned on
12:45 PM
Review security activity
10
How you sign in to Google
Make sure you can always access your Google Account by keeping this information up to date
2-Step Verification
On since 12:45 PM


myaccount.google.com/signinoptions/twosv?pli=1&rapt=AEjHL4PuRRjzqBLW5ecZtsU6Ga7Kay8978BoOZ3DtyHyHTXt6Bc2s_ayXobh_ecPgvzK-uCjB8Zxynx7nKAl6pnuTbv57rl_ISjW45hZAwmMh4m
Google Account
2-Step Verification
Google prompt
1 device
Authenticator
Add authenticator app
Phone number
084593 41537
Backup codes
Get backup codes
App passwords
App Passwords aren't recommended and are unnecessary in most cases. To help keep your account secure, use "Sign in with Google" to connect apps to your Google Account.
App passwords
2 App passwords
Turn off 2-Step Verification
Privacy Terms Help About





Click on Create Button



Copy Password and paste it in line : SENDER_PASSWORD = "hhxe atto vupb bnma"

Step 6: Run the Script
python send_emails.py

Optional Enhancements

1. **HTML Emails**

Replace:

MIMEText(body, "plain")

with:

MIMEText(body, "html")

Then use HTML templates.

---

2. Attachments

from email.mime.base import MIMEBase
from email import encoders

Attach PDFs/reports dynamically.

---

3. Schedule Automatic Sending

Windows Task Scheduler

Run daily/hourly automatically.

Linux Cron Job
0 9 * * * /usr/bin/python3 /path/send_emails.py

---

4. Send Only Pending Emails

df = df[df["Status"] == "Pending"]

---

5. Logging

Add:

import logging

Store delivery reports in log files.

Recommended Production Stack

For enterprise-grade systems:

- Use SMTP_SSL
- Store credentials in .env
- Use yagmail or smtplib
- Add retry logic
- Add rate limiting
- Use email APIs:
  - SendGrid
  - Mailgun
  - Amazon SES

Advanced Features You Can Add

- Personalized templates
- CC/BCC support
- Bulk campaign management
- Email open tracking
- Database integration
- GUI dashboard
- Multi-threaded sending
- Retry failed emails
- Auto-generate reports

Example Secure .env Configuration

Install:

pip install python-dotenv

Create .env

EMAIL=your_email@gmail.com
PASSWORD=your_app_password

Load securely:

```
from dotenv import load_dotenv
import os

load_dotenv()

SENDER_EMAIL = os.getenv("EMAIL")
SENDER_PASSWORD = os.getenv("PASSWORD")
```

Observation:

1. Email password generator.
2. Email successfully shared on mail output generated.

Conclusion:

Email automation was successfully implemented using Python.

Experiment No: 3

Title:

Perform web data extraction from a website (e.g., news headlines or product prices) and store the extracted data in Excel or CSV format.

Aim:

To Perform web data extraction from a website (e.g., news headlines or product prices) and store the extracted data in Excel or CSV format.

Objectives:

1. To understand web data extraction from website.
2. To understand how to store the extracted data in excel or csv format.
3. To understand how to used beautiful Soup and openpyxl libraries.

Requirement:

- Requests → fetch webpage
- Beautiful Soup → parse HTML
- pandas → save data
- openpyxl → export Excel

Theory:

Web Scraping is a technique through which you can automatically extract data from a website. It is a digital shortcut to manual copy-paste, but it is much faster and more accurate. In this, content is extracted using scripts or tools, such as text, prices, images, reviews, etc.

For example, imagine that you need the name, price and rating of 100 products from Amazon. Through Web Scraping, you can get this data in an Excel or CSV file in seconds. It is super helpful for researchers, marketers and coders.

The scraper understands the HTML structure of the page and picks the part it needs. Then, that data is saved in a file or database. With this, you can easily do the work of analysis, automation and reporting.

Types of Web Scrapers

Not every web scraper works the same way. Web Scraping tools are designed differently based on their use case, user needs, and technical setup. Let us understand four common categories in which you find web scrapers.

Self-built or Pre-built

You can build a web scraper yourself or use a pre-built (ready-made) tool. Self-built scrapers are used when you need a completely custom solution. To build them, programming knowledge is required like Python, HTML, CSS, XPath, etc. These are flexible but can be time-consuming.

Pre-built scrapers, on the other hand, are ready to go. All you have to do is install them and start scraping data. They have many extra features like:

1) Schedule scraping

2) Export output to Excel, JSON or Google Sheets

3) Get notifications when scraping is complete.

If you are a beginner, pre-built tools are best.

Browser Extension vs Software

Web scrapers come in two forms: browser extensions and standalone software.

Browser extensions are simple and easy to use. You can install them directly in browsers like Chrome and Firefox. They are lightweight and can easily perform basic scraping tasks. However, browser extensions have some limitations. Like:

1) Limited access to advanced features

2) IP rotation is not possible.

3) Difficulty handling heavy scraping

Software-based scrapers, on the other hand, are more powerful. You have to install a program on the system. These tools can handle complex scraping workflows, such as login-based scraping, captcha bypass, and heavy data extraction.

They are a little technical but are more useful in the long term.

User Interface

The user interface makes a lot of difference.

Some scrapers are command-line based, and the code has to be written in the Matlab terminal. This is great for developers, but beginners may get confused. Some scrapers come with a proper visual interface where you can easily select data by clicking. This drag-and-drop UI is beginner-friendly. Some tools also provide helpful tips so you understand which feature does what.

Cloud vs Local

Another major difference is the location of the scraper, cloud-based or local-based. Local scrapers run on your computer. This means:

1) CPU and RAM are used by your system

2) If there is heavy scraping, the system can slow down

3) Internet usage can also be high

Cloud scrapers run on an external server. You just have to assign the task, and the server scrapes the data for you. You can do other work while the scraping is complete. Cloud tools can easily handle IP rotation and heavy workloads. These are ideal for large-scale or regular scraping needs.

Learn how to turn raw text into valuable data – start your Advanced Data Science Certification journey now!

How to Scrape the Web?

If you are thinking that Web Scraping is just a matter of running a code, then wait a bit! There are some important steps in this process which you should follow, whether you are a beginner or have some experience. Below, we have explained six simple steps which form the base of every scraping journey.



Step 1: Gather the URLs to Scrape

First of all, you have to decide from which website you want data. If you want customer reviews for books, then you will collect the URLs of sites like Goodreads, Amazon, or LibraryThing. These links are the first step of your scraping process. The more URLs you have, the more data you can extract.

Step 2: Inspect the Web Page Elements

Now, you have to see what and where that data is on that website. Right-click on the browser and choose “Inspect” or “View Page Source” option. From here, you will see the backend HTML code of that page, which contains the information you want to scrape.

Step 3: Select the Specific Data to Extract

When you inspect, the corresponding HTML tag of the front-end element is highlighted. For example, if you want the title, author, and rating of a book on Amazon, you will have to look for tags like or in which this data is stored. These tags help in telling the scraper what to extract.

Step 4: Develop the Scraping Script

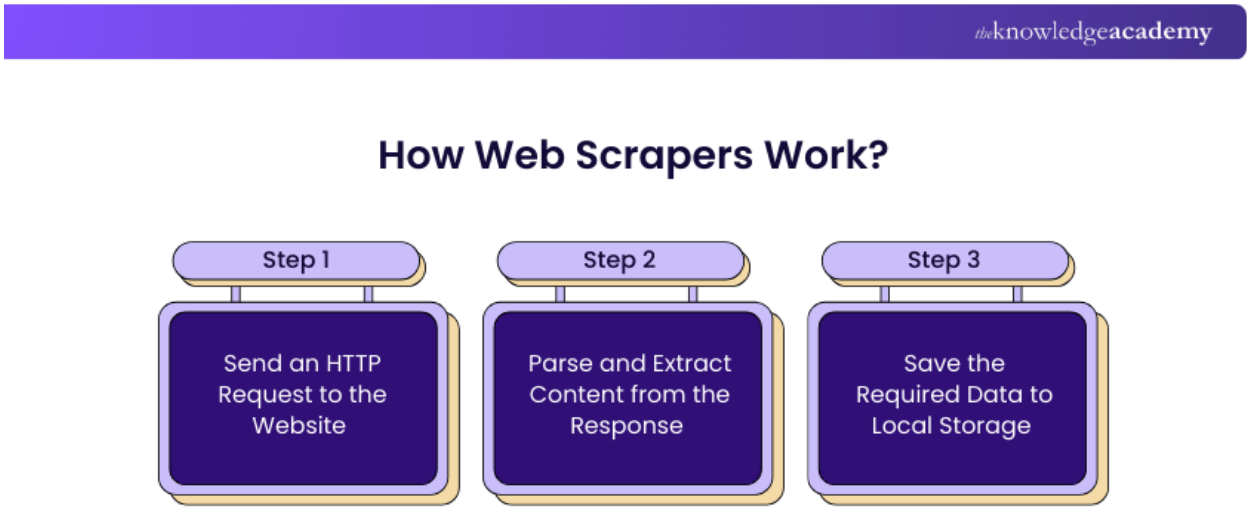
Now, it is time for some coding. You can write a scraping script in a language like Python using libraries like BeautifulSoup or Scrapy. This code tells the scraper what data to extract from which HTML tag. If you are a beginner, you can also use pre-built templates.

Step 5: Run Your Scraping Code

After the script is ready, run it. In this step, the scraper sends a request to the website, extracts the data, and processes it. If everything goes right, your desired data will be ready for analysis. If the site is slow or the request is blocked, it is important to be patient.

Step 6: Store the Extracted Data

After getting the data, saving it is equally important. You can save the scraped data in Excel, CSV or database format. Some people also use the Regex module to get clean output. After saving, you can analyse that data, create charts, or make business decisions.



Step 1: Send an HTTP Request to the Website

When we open a website in the browser, we send an HTTP request, meaning we digitally “knock” it. A web scraper does the same thing. It sends an HTTP request to the target website so that it can access the page’s data. If the website allows, the scraper moves forward.

Step 2: Parse and Extract Content from the Response

After the HTTP request is accepted, the website sends its data, usually in HTML or XML format. The scraper reads this code and "parses" it, that is, breaks it and understands where which data is stored. This step is important because it helps the scraper decide which content to extract from it, such as product name, price, ratings, or reviews.

Step 3: Save the Required Data to Local Storage

When the required data is found, the scraper saves that data, usually in Excel, CSV or JSON format. In this step, the data is converted into a structured format so that it can be used later for analysis, reporting or automation.

This process seems easy, but it is repeated several times, not just once. Every website has its own rules, and sometimes, sending too many requests can block the website. Hence, it is very important to use the scraping script in the right way.

Unlock insights from unstructured text with expert-led Predictive Analytics Course – join today!

Common Applications of Web Scraping

Nowadays, Web Scraping is being used not only by tech companies but in every industry. From small businesses to large corporations, everyone is collecting data and making smart decisions. Let's see some common uses:

1) Tracking Prices

Companies use Web Scraping to track product prices of their own and competitors'. As soon as the price of a product changes, the scraper catches that change. This helps companies set their pricing strategy so that they can make a profit and stay in the competition.

2) Conducting Market Analysis

Large-scale data is needed to understand market trends and customer behaviour. Web Scraping helps companies collect real-time data such as customer reviews, trending products, or buying patterns. This data becomes a gold mine for future planning and business growth.

3) Monitoring News Updates

News monitoring is very important for some businesses, especially those companies that depend heavily on media coverage. Web Scraping allows them to track every new article, blog, or mention. This makes it easier to manage reputation.

4) Performing Sentiment Analysis

What are people thinking about your product? Web Scraping lets you get real-time feedback from platforms like Facebook and Twitter. Whether it is positive or negative – sentiment analysis tells us how the audience is viewing your brand.

Tools for Web Scraping

You don't necessarily need to be a coding expert for Web Scraping. Python is a language with many tools and libraries that make this process very simple. Below are some popular tools that can be used by anyone from beginners to experts:

BeautifulSoup

BeautifulSoup is a Python library that helps you understand the structure of XML and HTML pages by reading them. You can easily extract data from any webpage, such as headlines, prices, reviews, etc. It is very friendly for beginners and makes things like searching and filtering easy.

Scrapy

Scrapy is a slightly advanced framework in Python, which is specially made for Web Scraping. It not only extracts data but also does automatic crawling. Once you define the rules, it automatically roams around the site and collects the data, just like a spider!

Pandas

Pandas are mainly used for data analysis, but if you use them with BeautifulSoup, Web Scraping is also possible. Its biggest advantage is that you can do the entire scraping and analysis in a single language (Python). It is also easy to get the output in CSV or Excel format.

Parsehub

Parsehub is for those people who are not comfortable with coding. This is a no-code tool where you can point-and-click to scrape data from a website. Just provide the URL, click on the data, and the scraper will do the job for you. The free version is available, but some features are available in the paid version.

Project Structure:

```
web_scraper/
│
├── scraper.py
├── output/
│   ├── data.csv
│   └── data.xlsx
├── requirements.txt
└── logs/
```

Program:

Step 1:

Install Required Libraries

Run:

pip install requests beautifulsoup4 pandas openpyxl

Example 1: Extract News Headlines

This example scrapes headlines from the BBC News website.

Create news_scraper.py

```
import requests
from bs4 import BeautifulSoup
import pandas as pd

# ====================================
# TARGET WEBSITE
# ====================================
url = "https://www.bbc.com/news"

# ====================================
# SEND REQUEST
# ====================================
headers = {
   "User-Agent": "Mozilla/5.0"
}
```

```
response = requests.get(url, headers=headers)

# ======================================
# PARSE HTML
# ======================================
soup = BeautifulSoup(response.text, "html.parser")

# ======================================
# EXTRACT HEADLINES
# ======================================
headlines = []

for item in soup.find_all("h2"):

    headline = item.get_text(strip=True)

    if headline and headline not in headlines:
        headlines.append(headline)

# ======================================
# STORE DATA IN DATAFRAME
# ======================================
df = pd.DataFrame({
    "Headline": headlines
})

# ======================================
# SAVE TO CSV
# ======================================


df.to_csv(r"C:\Users\Admin\Desktop\news_headlines.csv", index=False)
df.to_excel(r"C:\Users\Admin\Desktop\news_headlines.xlsx", index=False)

print("Data saved successfully.")
```

Run the Script
python3 news_scraper.py

Generated files:

news_headlines.csv
news_headlines.xlsx

Example 2: Extract Product Prices

Example scraping product titles and prices from a demo eCommerce site.

Create product_scraper.py

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
```

```
url = "https://books.toscrape.com/"

response = requests.get(url)

soup = BeautifulSoup(response.text, "html.parser")

products = []
prices = []

# Extract books
books = soup.find_all("article", class_="product_pod")

for book in books:

    title = book.h3.a["title"]

    price = book.find("p", class_="price_color").text

    products.append(title)
    prices.append(price)

# Create DataFrame
df = pd.DataFrame({
    "Product": products,
    "Price": prices
})

# Save data
df.to_csv(r"C:\Users\Admin\Desktop\products.csv", index=False)
df.to_excel(r"C:\Users\Admin\Desktop\products.xlsx", index=False)

print("Product data saved.")
```

Output Example

Product	Price
A Light in the Attic	£51.77
Tipping the Velvet	£53.74

Add Timestamp

```
from datetime import datetime

timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

Store scrape time in Excel.

Scrape Multiple Pages

Example:

```
for page in range(1, 6):

    url = f"https://books.toscrape.com/catalogue/page-{page}.html"
```

Handle Errors

```
try:
    response = requests.get(url)

    if response.status_code == 200:
        print("Success")
    else:
        print("Failed")

except Exception as e:
    print(e)
```

Advanced Features

For JavaScript-Based Websites

Install Selenium:

pip install selenium

Use browser automation when data is dynamically loaded.

Observation:

1. Web data successfully extraction.
2. Output excel file generated with extracted data.

Conclusion:

Web data extraction from a website and store the extracted data in Excel or CSV successfully executed.

Experiment No: 4

Title:

Implement an automated file organization system that sorts files into folders based on file type (PDF, images, documents).

Aim:

To Implement an automated file organization system that sorts files into folders based on file type (PDF, images, documents).

Objectives:

1. To understand how to automate file organization.
2. To understand how to sorts files into folders based on file type.
3. To understand how to used Watch dog and shutil libraries.

Requirement:

1. Desktop
2. Pycharm or Spyder
3. Libraries like Watch Dog and shutil

Theory:

Automated file sorting is a digital management approach that uses software to automatically organize, rename, and route files based on defined criteria or learned AI patterns. It transforms chaotic directories into searchable archives, saving time by eliminating manual drag-and-drop tasks across Mac and Windows. Automatic organization eliminates that time entirely. Files organize themselves as they arrive. Your workspace stays perfectly organized without any effort on your part. The hours you used to spend on file management become available for actual work.

Handling Different File Types

Automatic organization works across all file types, but it's worth understanding how the AI handles each:
Documents (PDFs, Word, Excel) get analyzed by content. The AI reads what's inside and organizes based on topic, purpose, and context. Invoices go with invoices. Contracts with contracts. Reports with reports.
Images get processed by object detection and OCR. The AI can tell property photos from product photos from random screenshots. It reads any text in images and uses that for organization context.
Audio and Video get transcribed. The AI knows what's being discussed and organizes based on content, not just filename. Meeting recordings end up with the correct project.
Generic Files without analyzable content get organized by metadata—filename, source, upload context, what else was uploaded with them. Even without content analysis, the AI uses available signals to make intelligent placement decisions.

Program:

Step 1: Install Python (if needed)

Verify:

python3 --version

Step 2: Create Python Script

Create:

file_organizer.py

Step 3: Complete Automation Script

```
import os
import shutil

# ========================================
# TARGET DIRECTORY
# ========================================

SOURCE_DIR = "C:/Users/Admin/Pictures"

# Example:
# SOURCE_DIR = "/home/user/Downloads"
# SOURCE_DIR = "C:/Users/YourName/Downloads"

# ========================================
# FILE TYPE CATEGORIES
# ========================================

FILE_CATEGORIES = {

    "PDFs": [".pdf"],

    "Images": [
        ".jpg", ".jpeg", ".png",
        ".gif", ".bmp", ".webp"
    ],

    "Documents": [
        ".doc", ".docx",
        ".txt", ".ppt", ".pptx",
        ".xls", ".xlsx", ".csv"
    ],

    "Videos": [
        ".mp4", ".mkv",
        ".avi", ".mov"
    ],

    "Music": [
```

```
        ".mp3", ".wav"
    ],

    "Archives": [
        ".zip", ".rar", ".7z"
    ]
}

# ========================================
# CREATE FOLDERS
# ========================================

for category in FILE_CATEGORIES.keys():

    folder_path = os.path.join(SOURCE_DIR, category)

    if not os.path.exists(folder_path):
        os.makedirs(folder_path)

# ========================================
# ORGANIZE FILES
# ========================================

for filename in os.listdir(SOURCE_DIR):

    file_path = os.path.join(SOURCE_DIR, filename)

    # Skip directories
    if os.path.isdir(file_path):
        continue

    # Get extension
    _, extension = os.path.splitext(filename)

    extension = extension.lower()

    moved = False

    # Match category
    for category, extensions in FILE_CATEGORIES.items():

        if extension in extensions:

            destination = os.path.join(
                SOURCE_DIR,
                category,
                filename
            )

            shutil.move(file_path, destination)

            print(f"Moved: {filename} -> {category}")

            moved = True
            break
```

```
    # Optional: Uncategorized files
    if not moved:

        other_folder = os.path.join(SOURCE_DIR, "Others")

        if not os.path.exists(other_folder):
            os.makedirs(other_folder)

        destination = os.path.join(other_folder, filename)

        shutil.move(file_path, destination)

        print(f"Moved: {filename} -> Others")

print("File organization completed.")
```

Step 4: Set Your Folder Path

Replace:

SOURCE_DIR = "/path/to/your/folder"

Examples:

Windows

```
SOURCE_DIR = "C:/Users/John/Downloads"
```

Linux / Codeanywhere

```
SOURCE_DIR = "/home/cabox/workspace"
```

Step 5: Run the Script

```
python3 file_organizer.py
```

Example Before

```
Downloads/
│
├── photo.jpg
├── report.pdf
├── music.mp3
└── data.xlsx
```

Example After

```
Downloads/
│
├── Images/
│   └── photo.jpg
│
├── PDFs/
│   └── report.pdf
│
├── Music/
│   └── music.mp3
```

```
├── Documents/
│   └── data.xlsx
```

Observation:

1. File management successfully organize.

Conclusion:

an automated file organization system that sorts files into folders based on file type successfully implemented.

Advanced Version: Organize by Date

You can also sort files by:

- Year
- Month
- Creation date

Example:

Images/2026/May/

Automatic Real-Time Monitoring

Install watchdog:

pip install watchdog

This allows:

✓ Auto-detect new files
✓ Organize instantly
✓ Continuous monitoring

Example Real-Time Automation
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler

Schedule Automatic Runs
Linux Cron Job
*/10 * * * * python3 /path/file_organizer.py

Runs every 10 minutes.

Windows Task Scheduler

You can schedule:

- On startup
- Daily
- Every hour

Add Duplicate File Handling

Prevent overwrite:

```
if os.path.exists(destination):
   print("File already exists")
```

Experiment No: 5

Title:

Automate attendance data processing by calculating attendance percentage and generating a summary report from a dataset.

Aim:

To Automate attendance data processing by calculating attendance percentage and generating a summary report from a dataset.

Objectives:

4. To understand how to automate attendance.
5. To understand how to process data.
6. To understand how to generate a system summary report.

Requirement:

1. **Desktop**
2. **IDE**
3. **Libraries**

Theory:

Attendance management is an important task in educational institutions and organizations for monitoring participation and performance. Manual calculation of attendance percentages is time-consuming and prone to errors, especially when dealing with large datasets. Automation helps process attendance records efficiently, accurately, and quickly.

In this practical, attendance data is stored in a dataset containing information such as student names, roll numbers, total classes conducted, and classes attended. Using a programming language or data-processing tool, the dataset is analyzed to calculate the attendance percentage for each student.

The attendance percentage is calculated using the formula:

Attendance Percentage=Classes Attended/Total Classes Conducted×100

After calculation, the system generates a summary report that displays important information such as student attendance percentages, average attendance, highest and lowest attendance records, and students who fall below the required attendance threshold. Data processing techniques such as filtering, sorting, aggregation, and report generation are used to obtain meaningful insights from the attendance dataset.

Automation of attendance processing improves efficiency, reduces human errors, saves time, and enables better decision-making. It also helps administrators and teachers quickly identify students with low attendance and take appropriate corrective actions. This practical demonstrates the application of data analysis and automation techniques in solving real-world administrative problems.

Example Dataset

Create attendance.xlsx

Student_ID	Name	Total_Classes	Attended
101	John	50	45
102	Jane	50	38
103	Alex	50	25

Step 1: Install Required Libraries
pip install pandas openpyxl

Step 2: Create Python Script

Create:

attendance_processor.py

Step 3: Complete Automation Script

```
import pandas as pd
import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart

import os
print(os.getcwd())

# ==========================================
# LOAD DATASET
# ==========================================

file_path = (r"C:\Users\Admin\Documents\Practical No5_Automated attendance
generator\attendance.xlsx")

df = pd.read_excel(file_path)

# ==========================================
# CALCULATE ATTENDANCE PERCENTAGE
# ==========================================

df["Attendance_Percentage"] = (
    df["Attended"] / df["Total_Classes"]
) * 100

# Round values
df["Attendance_Percentage"] = (
    df["Attendance_Percentage"].round(2)
)

# ==========================================
# ATTENDANCE STATUS
```

```
# ==========================================

def attendance_status(percent):

    if percent >= 75:
        return "Eligible"

    return "Low Attendance"

df["Status"] = df["Attendance_Percentage"].apply(
    attendance_status
)

# ==========================================
# SUMMARY REPORT
# ==========================================

total_students = len(df)

eligible_students = len(
    df[df["Attendance_Percentage"] >= 75]
)

low_attendance_students = len(
    df[df["Attendance_Percentage"] < 75]
)

average_attendance = (
    df["Attendance_Percentage"].mean().round(2)
)

# ==========================================
# DISPLAY SUMMARY
# ==========================================

print("\n===== ATTENDANCE SUMMARY =====")

print(f"Total Students: {total_students}")

print(f"Eligible Students: {eligible_students}")

print(f"Low Attendance Students: "
      f"{low_attendance_students}")

print(f"Average Attendance: "
      f"{average_attendance}%")

# ==========================================
# SAVE UPDATED REPORT
# ==========================================
output_file = "attendance_report.xlsx"

try:
    df.to_excel(output_file, index=False)
    print(f"\nExcel Report saved as: {output_file}")
```

```
except PermissionError:
    print("Error: Close attendance_report.xlsx if it is open.")

# ==========================================
# SAVE CSV REPORT
# ==========================================
df.to_excel("attendance_report_v2.xlsx", index=False)
df.to_csv("attendance_report_v2.csv", index=False)



# ==========================================
# GENERATE ATTENDANCE CHART
# ==========================================

import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))

plt.bar(
    df["Name"],
    df["Attendance_Percentage"],
    color="skyblue"
)

plt.xlabel("Students")
plt.ylabel("Attendance Percentage")
plt.title("Attendance Report")

# Optional: Show percentage values on bars
for i, value in enumerate(df["Attendance_Percentage"]):
    plt.text(i, value + 1, f"{value}%", ha="center")

plt.tight_layout()

plt.savefig("attendance_chart.png")

print("Chart saved as: attendance_chart.png")

plt.show()
import os
SENDER_PASSWORD = os.getenv("GMAIL_APP_PASSWORD")



from pathlib import Path

# ========================
# EMAIL CONFIGURATION
# ========================
SMTP_SERVER = "smtp.gmail.com"
SMTP_PORT = 587

SENDER_EMAIL = "dhokevidya27@gmail.com"
SENDER_PASSWORD = "wwqh hgpv mkrx wkjc"
```

```
SUBJECT = "Automated Attendance Notification"

# ==========================
# LOAD EXCEL FILE
# ==========================
excel_file = (r"C:\Users\Admin\Documents\Practical No5_Automated attendance
generator\attendance_report.xlsx")
df = pd.read_excel(excel_file)

# ==========================
# FILTER LOW ATTENDANCE STUDENTS
# ==========================
df = pd.read_excel(excel_file)

df["Attendance_Percentage"] = (
    df["Attended"] / df["Total_Classes"]
) * 100

df["Attendance_Percentage"] = df["Attendance_Percentage"].round(2)
low_students = df[df["Attendance_Percentage"] < 75]

# ==========================
# LOAD EMAIL TEMPLATE
# ==========================
template_path = Path("email_template.txt")
email_template = template_path.read_text()

# ==========================
# CONNECT SMTP SERVER
# ==========================
server = smtplib.SMTP(SMTP_SERVER, SMTP_PORT)
server.starttls()
server.login(SENDER_EMAIL, SENDER_PASSWORD)

# ==========================
# SEND EMAILS ONLY TO LOW STUDENTS
# ==========================
for index, row in low_students.iterrows():

    recipient_name = row["Name"]
    recipient_email = row["Email"]
    recipient_status = row["Status"]

    body = email_template.format(
        name=recipient_name,
        status=recipient_status
    )

    message = MIMEMultipart()
    message["From"] = SENDER_EMAIL
    message["To"] = recipient_email
    message["Subject"] = SUBJECT

    message.attach(MIMEText(body, "plain"))
```

```
    try:
        server.sendmail(
            SENDER_EMAIL,
            recipient_email,
            message.as_string()
        )

        print(f"Email sent to {recipient_email}")

    except Exception as e:
        print(f"Failed for {recipient_email}: {e}")

server.quit()
```

Step 4: Run the Script
python3 attendance_processor.py

Example Output
===== ATTENDANCE SUMMARY =====

Total Students: 3
Eligible Students: 2
Low Attendance Students: 1
Average Attendance: 72.67%

Report saved as: attendance_report.xlsx

Generated Excel Report

Student_ID	Name	Attendance_Percentage	Status
101	John	90.00	Eligible
102	Jane	76.00	Eligible
103	Alex	50.00	Low Attendance

Observation:

1. **Observe the how automatically system generate the attendance**

Conclusion:

Automate attendance data processing by calculating attendance percentage and generating a summary report from a dataset.

Experiment No: 6

Title:

Perform automated data cleaning by handling missing values, removing duplicate records, and generating a cleaned dataset.

Aim:

To Perform automated data cleaning by handling missing values, removing duplicate records, and generating a cleaned dataset.

Objectives:

To understand how to automate data cleaning.
To understand how to removing duplicate record.
To understand how to generate a generating a cleaned dateset.

Requirement:

Desktop
IDE
Libraries

Theory:

Data cleaning is a crucial step in the data preprocessing phase of data science and machine learning. Real-world datasets often contain inconsistencies such as missing values, duplicate entries, incorrect formats, and noisy data. These issues can significantly affect the performance of analytical models and lead to incorrect conclusions. Therefore, automated data cleaning helps in improving data quality and ensuring reliable analysis.

1. Handling Missing Values:

Missing values occur when no data value is stored for a variable in an observation. They may arise due to data entry errors, sensor failures, or incomplete surveys.

Common techniques:

- **Deletion:**
  - Remove rows or columns containing missing values.
  - Suitable when missing data is minimal.
- **Imputation:**
  - Replace missing values with statistical measures such as:
    - Mean (for numerical data)
    - Median (for skewed data)
    - Mode (for categorical data)
- **Forward/Backward Fill:**
  - Common in time-series data where previous or next values are used.

Proper handling ensures that datasets remain consistent without losing significant information.

2. Removing Duplicate Records:

Duplicate records are repeated entries in a dataset. They may occur due to merging datasets, human error, or system glitches.

Effects of duplicates:

- Bias in analysis
- Overestimation of results
- Increased storage and processing cost

Methods:

- Identify duplicates using key attributes or entire rows.
- Remove them using functions that retain only unique records.
- Keep first/last occurrence depending on requirement.

Removing duplicates ensures data integrity and prevents skewed results.

3. Generating a Cleaned Dataset:

After handling missing values and removing duplicates, the dataset becomes clean and consistent.

Steps involved:

1. Load raw dataset
2. Identify and treat missing values
3. Detect and remove duplicates
4. Validate data consistency
5. Save the cleaned dataset in a structured format (CSV, Excel, etc.)

A cleaned dataset is essential for accurate data analysis, visualization, and machine learning model training.

Observation:

1. Handle missing value
2. Remove duplicate row
3. Standardize data
4. Clean invalid entries
5. Generate cleaned Excel/CSV files
6. Create a cleaning summary report

Conclusion:

Data cleaning is a fundamental pre-processing step that improves the quality and reliability of data. By automating processes like handling missing values and removing duplicates, we ensure efficiency, consistency, and readiness of data for further analysis.

Program:

Example Dataset

Suppose raw_data.csv contains:

ID	Name	Age	Email
1	John	25	john@email.com
2	Jane	NaN	jane@email.com
3	John	25	john@email.com
4	Alex	30	NaN

Problems:

- Missing values
- Duplicate records
- Inconsistent data

Step 1: Install Required Libraries
pip install pandas openpyxl

Step 2: Create Python Script

Create:

data_cleaning.py

Step 3: Complete Automation Script

```
import sys
print(sys.executable)
import pandas as pd

import matplotlib.pyplot as plt
# ========================================
# LOAD DATASET
# ========================================

file_path = (r"C:\Users\Admin\Documents\Practical No6
Datacleaning_RemovingDuplicate\raw_data.csv")

df = pd.read_csv(file_path)

print("Original Dataset:")
print(df)

# ========================================
# DATA CLEANING SUMMARY
# ========================================

original_rows = len(df)
```

```
# ==========================================
# HANDLE MISSING VALUES
# ==========================================

# Fill missing Age with average age
if "Age" in df.columns:

    average_age = df["Age"].mean()

    df["Age"] = df["Age"].fillna(
        average_age
    )

# Fill missing Email with placeholder
if "Email" in df.columns:

    df["Email"] = df["Email"].fillna(
        "not_provided@example.com"
    )

# ==========================================
# REMOVE DUPLICATES
# ==========================================

duplicates_before = df.duplicated().sum()

df = df.drop_duplicates()

duplicates_removed = duplicates_before

# ==========================================
# CLEAN TEXT DATA
# ==========================================

if "Name" in df.columns:

    df["Name"] = (
        df["Name"]
        .str.strip()
        .str.title()
    )

# ==========================================
# REMOVE EMPTY ROWS
# ==========================================

df = df.dropna(how="all")

# ==========================================
# SUMMARY REPORT
# ==========================================

cleaned_rows = len(df)

missing_values_after = (
```

```
    df.isnull().sum().sum()
)

# ========================================
# DISPLAY REPORT
# ========================================

print("\n===== DATA CLEANING REPORT =====")

print(f"Original Rows: {original_rows}")

print(f"Cleaned Rows: {cleaned_rows}")

print(f"Duplicates Removed: "
      f"{duplicates_removed}")

print(f"Remaining Missing Values: "
      f"{missing_values_after}")

# ========================================
# SAVE CLEANED DATASET
# ========================================

clean_csv = "cleaned_data.csv"

clean_excel = "cleaned_data.xlsx"

df.to_csv(clean_csv, index=False)

df.to_excel(clean_excel, index=False)

print("\nCleaned dataset saved.")
```

Step 4: Run the Script
python3 data_cleaning.py

Example Output
===== DATA CLEANING REPORT =====

Original Rows: 4
Cleaned Rows: 3
Duplicates Removed: 1
Remaining Missing Values: 0

Generated Cleaned Dataset

ID	Name	Age	Email
1	John	25	john@email.com
2	Jane	26.67	jane@email.com
4	Alex	30	not_provided@example.com

Additional Cleaning Operations

Remove Invalid Emails

```
df = df[df["Email"].str.contains("@")]
```

Remove Negative Values

```
df = df[df["Age"] >= 0]
```

Convert Data Types

```
df["Age"] = df["Age"].astype(int)
```

Rename Columns

```
df.columns = df.columns.str.strip()
```

Optional: Generate Cleaning Log File

```
with open("cleaning_log.txt", "w") as f:

    f.write("Data Cleaning Report\n")
    f.write(f"Rows cleaned: {cleaned_rows}\n")
```

Optional: Visualize Missing Data

Install:

```
pip install missingno matplotlib
```

Example:

```
import missingno as msno
import matplotlib.pyplot as plt

msno.matrix(df)

plt.show()
```

```
data_cleaning_project/
│
├── raw_data.csv
├── data_cleaning.py
├── cleaned_data.csv
├── cleaned_data.xlsx
├── cleaning_log.txt
└── reports/
```

Example 1: Student Marks Summary Report

Sample Dataset (student_marks.csv)

Student	Math	Science	English
John	85	78	92
Jane	90	88	95
Alex	70	65	80

Step 1: Install Libraries
pip install pandas openpyxl matplotlib

Step 2: Create Python Script

Create:

student_report.py

Step 3: Complete Automation Script

```
import pandas as pd

# ========================================
# LOAD DATASET
# ========================================

file_path = "student_marks.csv"

df = pd.read_csv(file_path)

# ========================================
# CALCULATE TOTAL & AVERAGE
# ========================================

subjects = ["Math", "Science", "English"]

df["Total"] = df[subjects].sum(axis=1)

df["Average"] = (
    df[subjects].mean(axis=1).round(2)
)

# ========================================
# ASSIGN GRADES
# ========================================

def assign_grade(avg):

    if avg >= 90:
        return "A"

    elif avg >= 75:
        return "B"

    elif avg >= 60:
        return "C"

    return "D"

df["Grade"] = df["Average"].apply(assign_grade)

# ========================================
# SUMMARY STATISTICS
# ========================================
```

```
class_average = round(
    df["Average"].mean(),
    2
)

highest_score = df["Total"].max()

topper = df.loc[
    df["Total"].idxmax(),
    "Student"
]

# ========================================
# DISPLAY REPORT
# ========================================

print("\n===== STUDENT SUMMARY REPORT =====")

print(df)

print("\n===== CLASS STATISTICS =====")

print(f"Class Average: {class_average}")

print(f"Highest Score: {highest_score}")

print(f"Topper: {topper}")

# ========================================
# EXPORT REPORT
# ========================================

output_file = "student_summary_report.xlsx"

df.to_excel(output_file, index=False)

print(f"\nReport saved as: {output_file}")
```

Run the Script
python3 student_report.py

Example Output
===== CLASS STATISTICS =====

Class Average: 82.56
Highest Score: 273
Topper: Jane

Generated Report

Student	Total	Average	Grade
John	255	85.00	B

Student	Total	Average	Grade
Jane	273	91.00	A
Alex	215	71.67	C

Example 2: Sales Summary Report

Sample Dataset (sales.csv)

Product	Quantity	Price
Laptop	5	50000
Mouse	20	500
Keyboard	10	1500

Sales Report Script

```
import pandas as pd

df = pd.read_csv("sales.csv")

# Calculate total sales
df["Total_Sales"] = (
    df["Quantity"] * df["Price"]
)

# Summary
total_revenue = df["Total_Sales"].sum()

best_product = df.loc[
    df["Total_Sales"].idxmax(),
    "Product"
]

average_sales = round(
    df["Total_Sales"].mean(),
    2
)

# Display
print("\n===== SALES SUMMARY =====")

print(df)

print(f"\nTotal Revenue: ₹{total_revenue}")

print(f'Best Selling Product: {best_product}")

print(f"Average Sales: ₹{average_sales}")

# Export
df.to_excel(
    "sales_summary_report.xlsx",
    index=False
)
```

Optional: Generate Charts

Example bar chart:

```
import matplotlib.pyplot as plt

plt.bar(
    df["Student"],
    df["Average"]
)

plt.xlabel("Students")
plt.ylabel("Average Marks")
plt.title("Student Performance")

plt.savefig("student_chart.png")

plt.show()
```

Optional: Create PDF Reports

Install:

```
pip install reportlab
```

Generate professional PDF summaries.

Optional: Automated Email Reports

You can automatically send generated reports via email using SMTP.

Recommended Project Structure

```
report_automation/
│
├── datasets/
│   ├── student_marks.csv
│   └── sales.csv
│
├── reports/
│   ├── student_summary_report.xlsx
│   ├── sales_summary_report.xlsx
│   └── charts/
│
├── scripts/
│   ├── student_report.py
│   └── sales_report.py
│
└── logs/
```

Advanced Features

Extend the system with:

- Dashboard generation
- AI-powered insights
- Scheduled reports
- Database integration
- Web dashboards using Flask/Django
- Interactive charts with Plotly
- Power BI integration

Experiment No: 7

Title:
Generate automated summary reports from a dataset (student marks data) using Python.

Aim:
To generate automated summary reports (Excel, text and chart) from a student marks dataset using Python.

Objectives:

1. To understand report automation.
2. To understand libraries like pandas, openpyxl, matplotlib and pathlib.
3. To understand how to read a dataset, calculate statistics and generate a report automatically.

Theory:
An automated report system has three core layers: data extraction, data processing and report generation. The system reads a structured dataset (Excel/CSV), calculates statistics such as total, percentage, average, highest, lowest, pass percentage and topper, and then writes the results into output files (Excel summary, text report and chart) without manual effort. The same script can be re-run on new data at any time.

Requirement:

1. PyCharm IDE
2. Packages like pandas, openpyxl, matplotlib
3. Desktop

Project Structure

report_automation/

```
│
├── student_marks.xlsx
├── generate_report.py
├── requirements.txt
└── output/
    ├── summary_report.xlsx
    ├── summary_report.txt
    └── subject_average_chart.png
```

How to create a new .py file using Command Prompt

Path: C:\Users\Admin\Documents\MyPythonProject

Step 1: Open Command Prompt
Press Win + R → type cmd → Enter

Step 2: Go to your project folder

cd C:\Users\Admin\Documents\MyPythonProject

Step 3: Create a new Python file

- Option 1 (echo): echo. > generate_report.py
- Option 2 (type nul, better): type nul > generate_report.py
- Option 3 (create + open in Notepad): notepad generate_report.py

Step 4: Verify file created
Type dir. You will see generate_report.py in the list.

Procedure

Step 1: Install Dependencies
Create requirements.txt

pandas

openpyxl

matplotlib

Install:

pip install -r requirements.txt

Step 2: Create Dataset
Create student_marks.xlsx

Roll_No	Name	Maths	Science	English	Computer
1	John Doe	78	85	72	90
2	Jane Smith	92	88	81	95
3	Amit Patil	35	48	52	60
4	Sara Khan	66	70	74	68
5	Rahul Verma	55	42	38	61

Step 3: Python Automation Script
Create generate_report.py

python

```
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path
from datetime import datetime

# ==========================
# CONFIGURATION
# ==========================
INPUT_FILE = "student_marks.xlsx"
OUTPUT_DIR = Path("output")
OUTPUT_DIR.mkdir(exist_ok=True)
SUBJECTS = ["Maths", "Science", "English", "Computer"]
PASS_MARKS = 40

# ==========================
# LOAD DATASET
# ==========================
df = pd.read_excel(INPUT_FILE)

# ==========================
# PROCESS DATA
# ==========================
df["Total"] = df[SUBJECTS].sum(axis=1)
df["Percentage"] = (df["Total"] / (len(SUBJECTS) * 100) * 100).round(2)
df["Result"] = df[SUBJECTS].apply(
    lambda row: "Pass" if (row >= PASS_MARKS).all() else "Fail", axis=1
)

subject_stats = df[SUBJECTS].agg(["mean", "max", "min"]).round(2)
```

```
total_students = len(df)
passed = (df["Result"] == "Pass").sum()
failed = total_students - passed
pass_percent = round(passed / total_students * 100, 2)
class_average = df["Percentage"].mean().round(2)
topper = df.loc[df["Percentage"].idxmax()]

overall = pd.DataFrame({
    "Metric": ["Total Students", "Passed", "Failed", "Pass %",
               "Class Average %", "Topper", "Topper %"],
    "Value": [total_students, passed, failed, pass_percent,
              class_average, topper["Name"], topper["Percentage"]]
})

# ==========================
# SAVE EXCEL REPORT
# ==========================
excel_path = OUTPUT_DIR / "summary_report.xlsx"
with pd.ExcelWriter(excel_path, engine="openpyxl") as writer:
    df.to_excel(writer, sheet_name="Student Data", index=False)
    subject_stats.to_excel(writer, sheet_name="Subject Summary")
    overall.to_excel(writer, sheet_name="Overall Summary", index=False)

# ==========================
# SAVE CHART
# ==========================
plt.figure(figsize=(7, 4))
subject_stats.loc["mean"].plot(kind="bar", color="steelblue")
```

```
plt.title("Average Marks per Subject")
plt.ylabel("Average Marks")
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "subject_average_chart.png")
plt.close()

# ==========================
# SAVE TEXT REPORT
# ==========================
report = f"""STUDENT MARKS SUMMARY REPORT
Generated on: {datetime.now():%d-%m-%Y %H:%M}
-----------------------------------
Total Students  : {total_students}
Passed          : {passed}
Failed          : {failed}
Pass Percentage : {pass_percent}%
Class Average   : {class_average}%
Topper          : {topper['Name']} ({topper['Percentage']}%)

Subject-wise Statistics:
{subject_stats.to_string()}
"""
(OUTPUT_DIR / "summary_report.txt").write_text(report, encoding="utf-8")

print(report)
print("Reports generated in the 'output' folder.")
```

Step 4: Run the Script

```
python generate_report.py
```

Open the output folder to view the Excel report, text report and chart.

Optional Enhancements

1. **Use CSV data instead of Excel**

python

```
df = pd.read_csv("student_marks.csv")
```

2. **Add Grades**

python

```
def grade(p):
    return "A" if p >= 75 else "B" if p >= 60 else "C" if p >= 40 else "F"
df["Grade"] = df["Percentage"].apply(grade)
```

3. **Generate a PDF report:** use reportlab or fpdf.
4. **Email the report automatically:** attach the output file using MIMEBase and encoders (as in Experiment 2).
5. **Schedule Automatic Reports**
    - Windows Task Scheduler: run daily/weekly.
    - Linux Cron Job: 0 9 * * 1 /usr/bin/python3 /path/generate_report.py
6. **Show only failed students:** df[df["Result"] == "Fail"]
7. **Logging:** import logging and store run details in a log file.

Recommended Production Stack

- Use a database (MySQL/PostgreSQL) instead of Excel
- Store file paths and settings in .env
- Use seaborn/plotly for better charts
- Add error handling for missing files or columns
- Use Jinja2 for HTML report templates
- Use BI tools such as Power BI or Tableau for dashboards

Advanced Features You Can Add

- Subject-wise topper list
- Pie chart of pass/fail ratio
- Division-wise / class-wise comparison

- Rank generation
- Auto-email report to teachers
- GUI dashboard (Streamlit/Tkinter)
- Auto-generate PDF report cards
- Year-wise performance comparison

Example Secure Configuration (.env)

Install:

pip install python-dotenv

Create .env

INPUT_FILE=student_marks.xlsx

PASS_MARKS=40

Load:

python

from dotenv import load_dotenv

import os

load_dotenv()

INPUT_FILE = os.getenv("INPUT_FILE")

PASS_MARKS = int(os.getenv("PASS_MARKS"))

Observation:

1. The dataset was read successfully from the Excel file.
2. Total, percentage, result, subject statistics and topper were calculated automatically.
3. Excel report, text report and chart were generated in the output folder.

Conclusion:
Automated summary report generation was successfully implemented using Python.

Experiment No: 8

Title:

Analyze Automation Performance by Comparing Manual Processing Time with Automated Processing Time and Calculating Time Savings

Aim:

To analyze automation performance by comparing manual processing time with automated processing time and calculating the time savings achieved through automation.

Objectives:

1. To understand automation performance analysis.
2. To compare manual and automated processing times.
3. To calculate time savings and efficiency improvement.
4. To generate an automation performance report using Python.

Theory:

Automation is the process of performing tasks using software or technology with minimal human intervention. One important way to measure automation effectiveness is by comparing the time required to complete a task manually versus automatically.

The time saved can be calculated as:

```
Time Saved = Manual Time − Automated Time
```

The percentage time saving can be calculated as:

```
Percentage Time Saved =
(Time Saved / Manual Time) × 100
```

A higher percentage indicates better automation efficiency.

Requirements:

1. Python 3.x
2. PyCharm IDE or Google Colab
3. Desktop/Laptop

Project Structure

```
automation_analysis/

│
├── automation_performance.py
└── report.txt
```

How to Create Python File Using Command Prompt

Step 1: Open Command Prompt

Press:

```
Win + R
```

Type:

```
cmd
```

Press Enter.

Step 2: Go to Project Folder

```
cd C:\Users\Admin\Documents\MyPythonProject
```

Step 3: Create Python File

```
type nul > automation_performance.py
```

Step 4: Verify File Creation

```
dir
```

Output:

```
automation_performance.py
```

Procedure

Step 1: Open PyCharm or Google Colab

Create a new Python file named:

```
automation_performance.py
```

Step 2: Store Manual Processing Time

```
manual_time = 60
```

Instruction:

Store the time required to complete a task manually.

Example:

```
60 Minutes
```

Step 3: Store Automated Processing Time

```
automated_time = 15
```

Instruction:

Store the time required when automation is used.

Example:

```
15 Minutes
```

Step 4: Display Manual Time

```
print("Manual Processing Time :", manual_time, "minutes")
```

Instruction:

Display manual processing time.

Step 5: Display Automated Time

```
print("Automated Processing Time :", automated_time, "minutes")
```

Instruction:

Display automated processing time.

Step 6: Calculate Time Saved

```
time_saved = manual_time - automated_time
```

Instruction:

Subtract automated time from manual time.

Formula:

```
Time Saved = Manual Time − Automated Time
```

Calculation:

```
60 - 15 = 45 minutes
```

Step 7: Calculate Percentage Time Saved

```
percentage_saved = (time_saved / manual_time) * 100
```

Instruction:

Calculate percentage reduction in processing time.

Formula:

```
Percentage Time Saved
= (Time Saved / Manual Time) × 100
```

Calculation:

```
(45 / 60) × 100
= 75%
```

Step 8: Display Time Saved

```
print("Time Saved :", time_saved, "minutes")
```

Instruction:

Display total time saved.

Step 9: Display Percentage Saved

```
print("Percentage Time Saved :", round(percentage_saved, 2), "%")
```

Instruction:

Display percentage time savings.

Step 10: Generate Final Report

```
print("\n===== AUTOMATION PERFORMANCE REPORT =====")
print("Manual Time :", manual_time, "minutes")
print("Automated Time :", automated_time, "minutes")
print("Time Saved :", time_saved, "minutes")
print("Efficiency Gain :", round(percentage_saved, 2), "%")
```

Instruction:

Display complete automation performance report.

Complete Program

```
# ========================================
# AUTOMATION PERFORMANCE ANALYSIS
# ========================================

# Step 1: Store manual processing time
manual_time = 60

# Step 2: Store automated processing time
automated_time = 15

 # Step 3: Display manual processing time
 print("Manual Processing Time :", manual_time, "minutes")

 # Step 4: Display automated processing time
 print("Automated Processing Time :", automated_time, "minutes")

 # Step 5: Calculate time saved
 time_saved = manual_time - automated_time

 # Step 6: Calculate percentage time saving
 percentage_saved = (time_saved / manual_time) * 100

 # Step 7: Display time saved
 print("Time Saved :", time_saved, "minutes")

 # Step 8: Display percentage time saved
 print("Percentage Time Saved :", round(percentage_saved, 2), "%")

 # Step 9: Generate final report
 print("\n===== AUTOMATION PERFORMANCE REPORT =====")

 print("Manual Time :", manual_time, "minutes")

 print("Automated Time :", automated_time, "minutes")

 print("Time Saved :", time_saved, "minutes")

```

```
print("Efficiency Gain :", round(percentage_saved, 2), "%")

print("==========================================")
```

Output

```
Manual Processing Time : 60 minutes

Automated Processing Time : 15 minutes

Time Saved : 45 minutes

Percentage Time Saved : 75.0 %

===== AUTOMATION PERFORMANCE REPORT =====

 Manual Time : 60 minutes

 Automated Time : 15 minutes

 Time Saved : 45 minutes

 Efficiency Gain : 75.0 %

 ==========================================
```

Observation:

1. Manual processing time was 60 minutes.
2. Automated processing time was 15 minutes.
3. Time saved was 45 minutes.
4. Automation improved efficiency by 75%.

Result:

The automation performance was successfully analyzed by comparing manual and automated processing times and calculating the time saved.

Conclusion:

Automation significantly reduced the processing time. The analysis showed a time saving of 45 minutes and an efficiency improvement of 75%, demonstrating the effectiveness of automation in reducing effort and increasing productivity. [pract 9 bto 19 rpa | PDF]

Experiment No: 9

Title:

Develop a Simple Web Form Automation that Reads Input Data from an Excel Sheet and Automatically Fills and Submits the Form

Aim

To develop a simple web form automation system that reads input data from an Excel file and automatically fills and submits a form using Python.

Objectives

1. To understand web form automation.
2. To read data from an Excel file using Pandas.
3. To automate form filling using input data.
4. To simulate form submission automatically.
5. To reduce manual data entry effort.

Theory

Web Form Automation is the process of automatically entering data into web forms using software instead of manual typing.

In this experiment:

- Input data is stored in an Excel file.
- Python reads the Excel file.
- The data is automatically used to fill form fields.
- The form is submitted automatically.

Benefits:

- Saves time
- Reduces human errors
- Improves efficiency
- Handles repetitive tasks automatically

Requirements

1. Python 3.x
2. Google Colab / PyCharm
3. Pandas Library
4. OpenPyXL Library

Project Structure

Plain Text

web_form_automation/

```
│
├── students.xlsx
├── web_form_automation.py
└── output.txt
```

How to Create Python File Using Command Prompt

Step 1: Open Command Prompt

Press:

Plain Text

Win + R

Type:

Plain Text

cmd

Press Enter.

Step 2: Navigate to Project Folder

Plain Text

cd C:\Users\Admin\Documents\PythonProjects

Step 3: Create Python File

Plain Text

type nul > web_form_automation.py

Step 4: Verify File Creation

Plain Text

dir

Output:

Plain Text

web_form_automation.py

Procedure

Step 1: Import Required Library

Python

import pandas as pd

Instruction:

Import Pandas library for reading Excel files and handling data.

\

Step 2: Create Sample Student Data

Python

data = {

"Name": ["Amit", "Priya"],

"Email": ["amit@gmail.com", "priya@gmail.com"]

}

Instruction:

Create sample student data containing Name and Email.

Step 3: Convert Data into DataFrame

Python

df = pd.DataFrame(data)

Instruction:

Convert the dictionary data into a tabular DataFrame format.

Step 4: Save Data into Excel File

Python

df.to_excel("students.xlsx", index=False)

Instruction:

Create and save the Excel file automatically.

File Created:

Plain Text

students.xlsx

Step 5: Read Excel File

Python

students = pd.read_excel("students.xlsx")

Instruction:

Read data stored in the Excel sheet.

Step 6: Display Excel Data

Python

print(students)

``

Instruction:

Display the data read from the Excel file.

Output:

Plain Text

Name Email

0 Amit amit@gmail.com

1 Priya priya@gmail.com

Step 7: Read First Record

Python

for index, row in students.iterrows():

Instruction:

Read each row one by one from the Excel sheet.

Step 8: Start Form Filling Process

Python

print("\n----- Filling Form -----")

Instruction:

Display a message indicating the start of form automation.

Step 9: Fill Name Field

Python

print("Entering Name :", row["Name"])

Instruction:

Read Name from Excel and enter it into the form.

Example:

Plain Text

Amit

Step 10: Fill Email Field

Python

print("Entering Email :", row["Email"])

Instruction:

Read Email from Excel and enter it into the form.

Example:

Plain Text

amit@gmail.com

Step 11: Submit Form

Python

```
print("Submitting Form ...")
```

Instruction:

Simulate clicking the Submit button.

Step 12: Display Success Message

Python

```
print("Form Submitted Successfully")
```

Instruction:

Display confirmation of successful form submission.

Complete Program

Python

```
# ======================================
# WEB FORM AUTOMATION USING EXCEL DATA
# ======================================

# Step 1: Import pandas library
import pandas as pd

# Step 2: Create sample data
data = {
"Name": ["Amit", "Priya"],
"Email": ["amit@gmail.com", "priya@gmail.com"]
```

```
}

# Step 3: Convert data to DataFrame
df = pd.DataFrame(data)

# Step 4: Save data into Excel file
df.to_excel("students.xlsx", index=False)

# Step 5: Read Excel file
students = pd.read_excel("students.xlsx")

# Step 6: Display Excel data
print("Student Data")
print(students)

# Step 7: Read records one by one
for index, row in students.iterrows():

# Step 8: Start automation
print("\n----- Filling Form -----")

# Step 9: Fill Name field
print("Entering Name :", row["Name"])

# Step 10: Fill Email field
print("Entering Email :", row["Email"])

# Step 11: Submit form
print("Submitting Form ...")
```

```
# Step 12: Display success message
print("Form Submitted Successfully")
```

Output

```
Plain Text
Student Data

Name Email
0 Amit amit@gmail.com
1 Priya priya@gmail.com

----- Filling Form -----

Entering Name : Amit

Entering Email : amit@gmail.com

Submitting Form ...

Form Submitted Successfully

----- Filling Form -----

Entering Name : Priya

Entering Email : priya@gmail.com

Submitting Form ...
```

Form Submitted Successfully

Algorithm

1. Start the program.
2. Import Pandas library.
3. Create student data.
4. Convert data into DataFrame.
5. Save data into Excel file.
6. Read Excel file.
7. Display Excel data.
8. Read each row from Excel.
9. Fill Name field.
10. Fill Email field.
11. Submit form.
12. Display success message.
13. Stop the program.

Observation

1. Excel data was successfully created.
2. Data was successfully read from the Excel file.
3. Name and Email fields were automatically processed.
4. Form submission process was completed successfully.
5. Manual data entry effort was reduced.

Result

The web form automation system successfully read data from an Excel file and automatically filled and submitted the form.

Conclusion

The experiment successfully demonstrated web form automation using Python and Excel data. The automation reduced manual effort, improved efficiency, and simplified repetitive form-filling tasks.

Experiment No: 10

Title:

Create an Automation Workflow that Reads Data from a Dataset and Generates Visual Charts or Graphs for Analysis

Aim

To create an automation workflow that reads data from a dataset and automatically generates visual charts for data analysis using Python.

Objectives

1. To understand data visualization concepts.
2. To read data from a dataset.
3. To automate chart generation using Python.
4. To visualize data using graphs.
5. To analyze data effectively through charts.

Theory

Data Visualization is the graphical representation of data using charts and graphs. It helps users understand trends, comparisons, and patterns easily.

Automation Workflow in Data Visualization involves:

- Reading data automatically from a dataset.
- Processing the data.
- Generating charts without manual calculations.
- Displaying graphical insights for analysis.

Benefits:

- Easy data interpretation
- Faster analysis
- Improved decision-making
- Automated report generation

Requirements

1. Python 3.x
2. Google Colab / PyCharm
3. Pandas Library
4. Matplotlib Library

Project Structure

Plain Text

data_visualization/

│

├── chart_analysis.py

└── dataset.csv

How to Create Python File Using Command Prompt

Step 1: Open Command Prompt

Press:

Plain Text

Win + R

Type:

Plain Text

cmd

Press Enter.

Step 2: Navigate to Project Folder

Plain Text

cd C:\Users\Admin\Documents\PythonProjects

Step 3: Create Python File

Plain Text

type nul > chart_analysis.py

Step 4: Verify File Creation

Plain Text

dir

Output:

Plain Text

chart_analysis.py

Procedure

Step 1: Import Required Libraries

Python

import pandas as pd

import matplotlib.pyplot as plt

Instruction:

Import Pandas library for handling data and Matplotlib library for generating charts.

Step 2: Create Dataset

Python

data = {

"Student": ["Amit", "Priya", "Rahul", "Sneha"],

"Marks": [80, 95, 70, 88]

}

Instruction:

Create a dataset containing student names and marks.

Step 3: Convert Data into DataFrame

Python

df = pd.DataFrame(data)

Instruction:

Convert the dataset into a table format called DataFrame.

Step 4: Display Dataset

Python

print(df)

Instruction:

Display the dataset for verification.

Output:

Plain Text

Student Marks

0 Amit 80

1 Priya 95

2 Rahul 70

3 Sneha 88

Step 5: Create Bar Chart

Python

plt.bar(df["Student"], df["Marks"])

Instruction:

Create a bar chart using student names and marks.

Step 6: Add Chart Title

Python

plt.title("Student Marks Analysis")

Instruction:

Provide a meaningful title to the chart.

Step 7: Add X-Axis Label

Python

```
plt.xlabel("Student Name")
```

Instruction:

Label the horizontal (X) axis.

Step 8: Add Y-Axis Label

Python

```
plt.ylabel("Marks")
```

Instruction:

Label the vertical (Y) axis.

Step 9: Display Chart

Python

```
plt.show()
```

Instruction:

Display the generated chart on the screen.

Step 10: Display Success Message

Python

```
print("Chart Generated Successfully")
```

Instruction:

Display confirmation that chart generation is completed.

Complete Program

Python

```
# ========================================
# AUTOMATED CHART GENERATION WORKFLOW
# ========================================
```

```
# Step 1: Import required libraries
import pandas as pd
import matplotlib.pyplot as plt

# Step 2: Create dataset
data = {
"Student": ["Amit", "Priya", "Rahul", "Sneha"],
"Marks": [80, 95, 70, 88]
}

# Step 3: Convert data into DataFrame
df = pd.DataFrame(data)

# Step 4: Display dataset
print("Dataset:")
print(df)

# Step 5: Generate bar chart
plt.bar(df["Student"], df["Marks"])

# Step 6: Add chart title
plt.title("Student Marks Analysis")

# Step 7: Add X-axis label
plt.xlabel("Student Name")

# Step 8: Add Y-axis label
plt.ylabel("Marks")
```

```
# Step 9: Display chart
plt.show()

# Step 10: Display success message
print("Chart Generated Successfully")
```

Output

Plain Text

Dataset:

Student Marks

0 Amit 80

1 Priya 95

2 Rahul 70

3 Sneha 88

Chart Generated Successfully

Generated Chart

Plain Text

Student Marks Analysis

95 | ▉

90 | ▉

85 | ▉

80 | ▉▉

75 | ▉▉

70 | ▉▉▉

65 | ▉▉▉

Amit Priya Rahul Sneha

(Actual output will be displayed as a graphical Bar Chart.)

Algorithm

1. Start the program.
2. Import Pandas and Matplotlib libraries.
3. Create a dataset.
4. Convert dataset into a DataFrame.
5. Display the dataset.
6. Generate a bar chart.
7. Add chart title.
8. Add X-axis label.
9. Add Y-axis label.
10. Display the chart.
11. Display success message.
12. Stop the program.

Observation

1. The dataset was successfully created.
2. Student marks were read from the dataset.
3. A bar chart was generated automatically.
4. The chart provided a visual comparison of student marks.
5. Data analysis became easier through graphical representation.

Result

The automation workflow successfully read data from the dataset and generated a visual chart for analysis.

Conclusion

The experiment successfully demonstrated automated chart generation using Python. The system read data from a dataset and automatically created a visual chart, making data analysis simpler, faster, and more effective.